



RS6x_7x mmWave Radar Sample Reference Manual

Number of Pages: 34

Document Version: 1.2

Release Date: 2026-9-4



Revision History

Version	Revision Description	Date
1.2	Revised the description of presence point cloud detection method configuration in Section 4.2.7.2 Added the description of presence point cloud detection threshold configuration in Section 4.2.7.3 Revised the presence point cloud heap configuration description in Section 4.2.7.8 Revised the presence point cloud computation time in the typical performance specifications in Section 4.3.5	2026-09-03
1.1	Added the default configuration description for 6241 to Table 3-3.	2026-08-13
1.0	Initial release	2026-06-12

Table of Contents

Revision History	ii
Table of Contents	iii
List of Figures	v
List of Tables	vi
1 Purpose and Scope	1
2 Overview	2
3 Evaluating Samples Based on radar framework	3
3.1 Selecting a Sample for Evaluation	3
3.2 Setting Up the Hardware	4
3.3 Viewing Sample Output	4
4 Further Development with radar framework	7
4.1 Code Structure	7
4.2 Configuring radar framework	7
4.2.1 Functional Configuration Guide	8
4.2.2 Global radar framework Configuration	8
4.2.2.1 Globally Enabling the 1D Frame Structure	8
4.2.2.2 Globally Enabling the 2D Frame Structure	9
4.2.2.3 Configuring the Maximum Supported Number of Range FFT Points	9
4.2.2.4 Configuring the DataCube Buffer	9
4.2.2.5 Configuring Adaptive Gain	9
4.2.2.6 Reporting Frame Information	9
4.2.2.7 Enabling Heartbeat Reporting	9
4.2.2.8 Enabling the Watchdog	9
4.2.2.9 Enabling Task Monitor	9
4.2.3 1D Frame Structure Subfunction Configuration	10
4.2.3.1 Reporting 1D-DataCube	10
4.2.3.2 Heap Usage Configuration	10
4.2.4 2D Frame Structure Functional Configuration	10
4.2.4.1 Globally Enabling the 2D Frame Structure	10
4.2.4.2 Reporting 2D-DataCube	11
4.2.5 Obtaining the Range FFT of Interval0	11
4.2.6 Motion Point Cloud	12
4.2.6.1 Configuring Software CFAR	12
4.2.6.2 Configuring Motion Point Cloud Reporting	12
4.2.6.3 Configuring the Maximum Number of Motion Point Cloud CFAR Points	12
4.2.6.4 Motion Point Cloud Heap Configuration	12
4.2.7 Presence Point Cloud	12
4.2.7.1 Configuring the Maximum Number of Presence Point Cloud Points	13
4.2.7.2 Configuring the Presence Point Cloud Detection Method	13
4.2.7.3 Configuring the Presence Point Cloud Detection Threshold	13
4.2.7.4 Configuring the Number of Presence Point Cloud Range Bins	13
4.2.7.5 Configuring the Presence Point Cloud Range Decimation Factor	13
4.2.7.6 Configuring the Number of Buffered Presence Point Cloud Intervals	13
4.2.7.7 Configuring the Number of Presence Point Cloud Doppler FFT Points	14
4.2.7.8 Presence Point Cloud Heap Configuration	14
4.3 Application Configuration Based on radar framework	14
4.3.1 Waveform Configuration	14
4.3.1.1 MIMO Mode	15
4.3.1.2 Start Frequency	15
4.3.1.3 Frame Type	15
4.3.1.4 Number of Range FFT Points (log2)	15

4.3.1.5 Range Resolution	16
4.3.1.6 Number of Doppler FFT Points (log2)	16
4.3.1.7 Velocity Resolution	16
4.3.1.8 Number of Accumulated Chirps (log2)	16
4.3.1.9 FFT Mode	16
4.3.1.10 Static Clutter Convergence Mode	17
4.3.2 Point Cloud Performance Configuration Guide	17
4.3.3 Function Tailoring Guide	17
4.3.4 Typical Waveform Configurations	18
4.3.5 Typical Performance Metrics	18
4.4 Digital Signal Processing Framework	19
4.5 HIF Configuration Framework	20
5 [Q & A]	21
5.1 How Can Point Cloud Density Be Increased?	21
5.2 How Can More SRAM Be Made Available?	21
5.3 Why Does Point Cloud Output Stutter, and How Can Computation Time Be Reduced?	21
5.4 Are There Best Practices for Waveform Configuration?	21
5.5 What Are the Computation Timing Constraints?	22
5.6 How Can Detection of Weakly Reflecting Targets in the Radar Field of View Be Improved?	22
5.7 Why Are Presence Point Cloud False Detections Likely at Short Range?	23
5.8 Why Are There Many False Targets in Indoor Motion Point Clouds?	23
5.9 Why Might Configuration with RadarDebugTool Fail?	24
5.10 Why Are Spectra and Point Clouds Abnormal with Certain Chirp Accumulation Counts?	24
5.11 What Should Be Done When the Debug UART Prints malloc fail?	24
5.12 What Do the Error Messages Printed by the Debug UART Mean?	24
6 Appendix	25
6.1 Digital Signal Processing with ADC and 1D-DataCube Data	25
6.1.1 Velocity Compensation Coefficient	25
6.1.2 Angle Calibration Values	25
6.1.3 ADC Data Point Cloud Processing Flow	25

List of Figures

Figure 3-1 Point Cloud Display in the Host Tool	5
Figure 3-2 DataCube Display in the Host Tool	6
Figure 4-1 Configuration Parameter Override Logic.....	8
Figure 4-2 Data Path in the 1D Frame Structure	10
Figure 4-3 Data Path in the 2D Frame Structure	11
Figure 4-4 Waveform Structure	15
Figure 4-5 2D Digital Signal Processing Framework	19
Figure 4-6 HIF Configuration Framework Call Flow	20
Figure 5-1 Multipath Interference	23
Figure 5-2 "Shadow" False Targets	24
Figure 6-1 ADC Data Point Cloud Processing Flowchart	27

List of Tables

Table 3-1 Basic Functions of Sample Projects	3
Table 3-2 Application Scenarios of Sample Projects	3
Table 3-3 Default Hardware Settings	4
Table 4-1 Typical Computation Time	18
Table 4-2 Available SRAM	19

1 Purpose and Scope

This document describes in detail how to use and configure the radar-related Samples provided in the Possumic technology RS6x_7x series mmWave radar Software Development Kit (SDK). After gaining a thorough understanding of these Samples, developers can customize or optimize algorithms and add post-processing logic based on the Sample framework to accelerate product development.

This document supports all chips in the RS6x and RS7x series.

2 Overview

The Samples provided in the mmWave radar SDK are based on radar framework, a lightweight, configurable radar framework that provides the following radar application functions:

- Transmit DataCube data using a 1D or 2D frame structure and visualize it in RadarDebugTool;
- Transmit Analog-to-Digital Converter (ADC) data using a 1D or 2D frame structure and visualize it in RadarDebugTool;
- Report the range Fast Fourier Transform (FFT) data of the first Interval in a 2D frame structure and visualize it in RadarDebugTool;
- Calculate and report motion point clouds and visualize them in RadarDebugTool;
- Calculate and report presence point clouds and visualize them in RadarDebugTool;
- Configure basic waveform parameters through RadarDebugTool;
- Provide production test functions that can be tested using ProductTestingAnalysisTool;
- Provide sleep and wake-up functions: the Central Processing Unit (CPU) enters sleep after completing digital signal processing, with current consumption < 10 μ A;
- Call hook functions at the start and end of radar digital signal processing;
- Customize Host Interface (HIF) application-layer transmission and configuration protocols.

radar framework has passed stability and stress tests and includes a keep-alive mechanism. It can serve as a baseline for further development for production.

3 Evaluating Samples Based on radar framework

The SDK provides precompiled firmware files. Developers can select a firmware file that meets their requirements and download it to the development board. Use RadarDebugTool to quickly evaluate DataCube, motion point cloud, and presence point cloud performance based on radar framework, and use ProductTestingAnalysisTool to verify production test functions.

The firmware files are located at:

[/Software Kit/04 Image/Tracking \[board name\].img](#).

[/Software Kit/04 Image/Posture \[board name\].img](#).

[/Software Kit/04 Image/Sleep \[board name\].img](#).

[/Software Kit/04 Image/ReportDataCube1D \[board name\].img](#).

[/Software Kit/04 Image/ReportDataCube2D \[board name\].img](#).

For instructions on using RadarDebugTool and downloading firmware, refer to '[RS6x 7x mmWave Radar Development Quick Start Guide.pdf](#)'.

3.1 Selecting a Sample for Evaluation

The radar framework functions supported by each sample project are listed below. Firmware is named [Function]_[Development Board Model].img. Before downloading firmware, make sure that its target development board matches the model being used. If the board model is unclear, refer to '[RS6x 7x mmWave Radar Development Quick Start Guide.pdf](#)'. The specific Sample functions are described in Section 4.2.

Table 3-1 Basic Functions of Sample Projects

Project Name	1D-DataCube	2D-DataCube	Motion Point Cloud	Presence Point Cloud	Interval0 RFFT	Frame Period	Production Test	Low Power
ReportDataCube1D	√	×	×	×	×	10ms	×	×
ReportDataCube2D	×	√	×	×	×	100ms	×	×
Posture	×	×	√	√	×	50ms	√	×
Sleep	×	×	√	√	√	50ms	√	×
Tracking	×	×	√	×	×	50ms	√	√

Typical application scenarios for the Samples are listed in the following table:

Table 3-2 Application Scenarios of Sample Projects

	Application Scenario
ReportDataCube1D	The radar acts as a data generator using a 1D frame structure, supporting an Interval rate of up to 200 Hz. It sends data to the Host through Serial Peripheral Interface (SPI), and the Host performs subsequent digital signal processing.

	Application Scenario
ReportDataCube2D	The radar acts as a data generator using a 2D frame structure. It sends data to the Host through SPI, and the Host performs subsequent digital signal processing.
Posture	For short-range posture recognition applications and robotic platforms within 5 m. It generates short-range, high-density motion and presence point clouds and does not detect stationary targets.
Sleep	For medium-range, medium-density motion and presence point clouds within 10 m. It supports reporting the range FFT data of Interval0 for subsequent processing in breathing and heartbeat detection applications and does not detect stationary targets.
Tracking	For motion point clouds within 20 m in outdoor scenarios, with low-power support. It does not detect stationary targets.

3.2 Setting Up the Hardware

Sample firmware compiled for different development boards has different default communication interfaces. First, refer to the "Hardware Setup" section in ['RS6x 7x mmWave Radar Evaluation Quick Start Guide.pdf'](#) for definitions of the communication modes, and complete the hardware setup.

Table 3-3 Default Hardware Settings

	MRS6130_P1806	MRS6130_P1812	MRS6240_P2512/ MRS6241_P2828/ MRS6241_P2840
Communication Mode	DUAL-UART	UART-SPI	UART-SPI
Debug Output	UART1, 115200bps	UART, 115200bps	UART, 115200bps
HIF (Data Interface)	UART0, 1Mbps	SPI, 60Mbps	SPI, 60Mbps

Note: Place the radar on a stable platform when evaluating Sample functions. Moving the radar may cause excessive point cloud density and computation timeouts, affecting the evaluation. Unexpected platform vibration may cause false point cloud detections.

3.3 Viewing Sample Output

In RadarDebugTool, select point cloud, DataCube, or debug information for display. Right-click in the plotting window and select the data to display. Figure 3-1 shows the point cloud results of three Samples in typical application scenarios. The two plots on the left are 2D "X-Y" point cloud views, and the plot on the right is a 3D point cloud view:

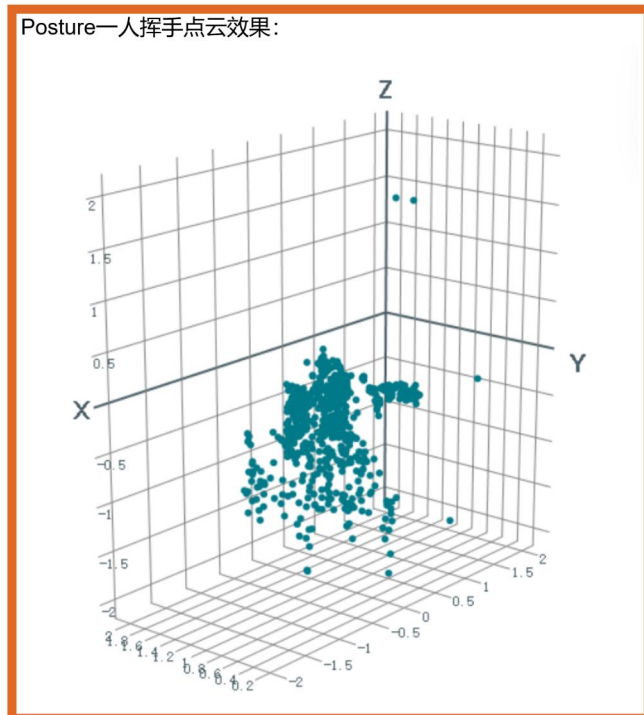
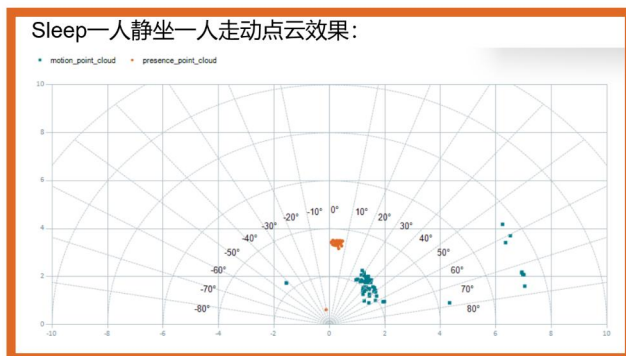
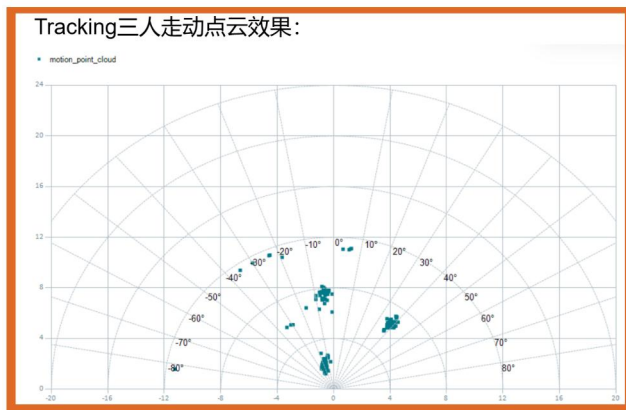


Figure 3-1 Point Cloud Display in the Host Tool

Figure 3-2 shows the DataCube data reported by the "ReportDataCube1D" and "ReportDataCube2D" Samples:



Figure 3-2 DataCube Display in the Host Tool

In addition to viewing Sample output, RadarDebugTool supports modifying frame configurations. For more information on using RadarDebugTool, refer to ['RS6x 7x RadarDebugTool User Manual.pdf'](#).

4 Further Development with radar framework

All features supported by radar framework can be implemented through static configuration.

Developers can select the project that most closely matches their requirements, modify its configuration, and compile firmware that meets their application requirements.

Developers can also register callback functions to obtain radar framework computation results for subsequent processing and add custom configurations to create production firmware with more features.

4.1 Code Structure

The radar framework code structure and key header files are shown in the following table:

<home>/Software_Kit/02_SDK/ project/mmwave	
__ common	
__ radar_framework	
__ inc	
__ radar_framework.h	/* radar framework APIs*/
__ radar_framework_default_config.h	/* Default function and heap configuration */
__ radar_framework_1D_frame_config.h	/* 1D frame structure functional configuration */
__ radar_framework_2D_frame_config.h	/* 2D frame structure functional configuration */
__ radar_framework_hif_config.h	/* HIF configuration */
__ radar_framework_Interval0_report_config.h	/* Interval_0 reporting configuration */
__ radar_framework_motion_pointcloud_config.h	/* Motion point cloud configuration */
__ radar_framework_presence_pointcloud_config.h	/* Presence point cloud configuration*/
__ radar_framework_soc_config.h	/* SOC interface configuration */
__ src	/* radar framework source code */
__ radar_framework.c	/* radar framework main flow*/
__ radar_framework_hif_handler.c	/* HIF configuration callback*/
__ radar_framework_report.c	/* HIF-based data reporting */
__ radar_signal_process_1d.c	/* 1D frame structure callback processing flow */
__ radar_signal_process_2d.c	/* 2D frame structure callback processing flow */
__ mmwave	/* DataCube reporting sample project */
__ mmwave_application	/* Point cloud application sample project */
<home>/Software_Kit/02_SDK/ __ subsys	
__ hif	
__ hif.h	/* Main HIF header file */
__ mmw	
__ mmw_algorithm	/* Point cloud processing unit module */
__ mmw_application	
__ mmw_app_pointcloud_config.h	/* Point cloud computation subsystem configuration
header */	
__ mmw_app_micro_pointcloud.c	/* Presence point cloud implementation source */
__ mmw_app_pointcloud.c	/* Motion point cloud implementation source */

4.2 Configuring radar framework

radar framework supports functional and performance configuration. Functional configuration enables or disables modules by overriding the macros in [project/common/radar_framework/inc/radar_framework_default_config.h](#) in the user project's prj_config.h. Performance configuration overrides the default-value macros in [subsys/mmwave_app_pointcloud_config.h](#) in the user project's prj_config.h.

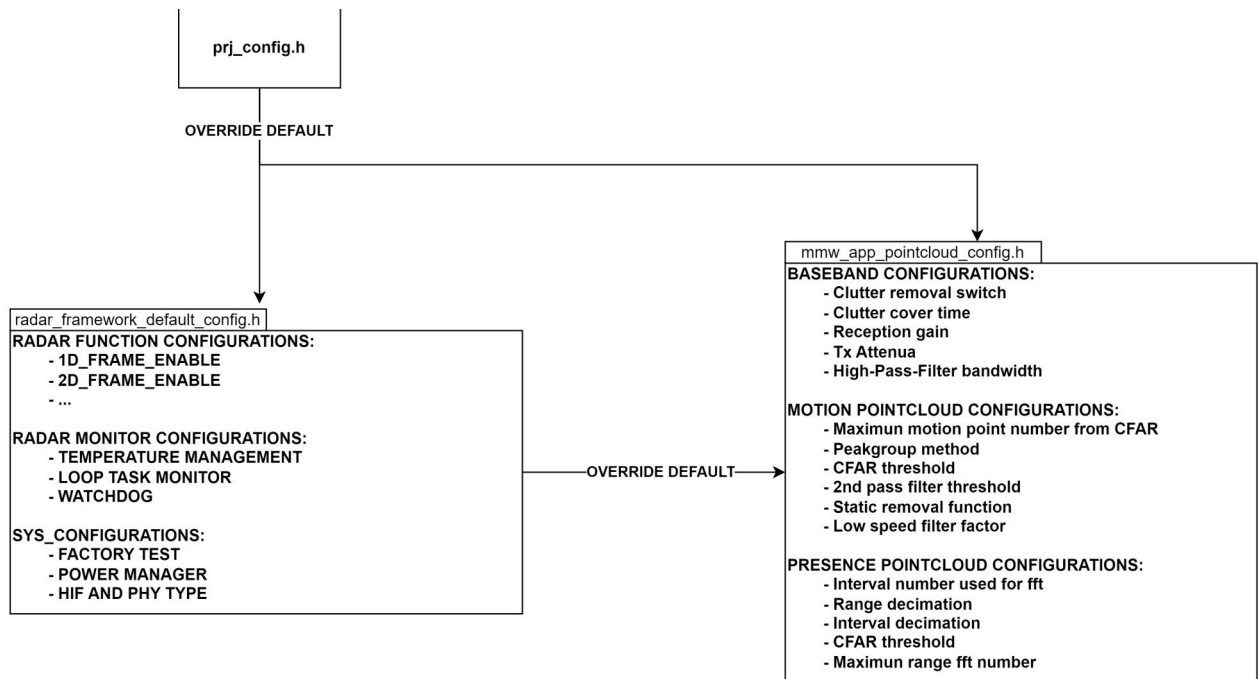


Figure 4-1 Configuration Parameter Override Logic

This section describes the functional configuration in '[project/common/radar_framework/inc/radar_framework_default_config.h](#)'. Point cloud performance configuration is located in `subsys/mmwave_app_pointcloud_config.h`. Refer to '[RS6x 7x mmWave Radar Point Cloud Processing Reference Manual.pdf](#)'.

4.2.1 Functional Configuration Guide

The default functional configuration templates are located in [project/common/radar_framework/inc](#). radar framework divides functional configuration into the following eight parts:

1. Global radar framework configuration — `radar_framework_default_config.h`;
2. 1D frame structure functional configuration — `radar_framework_1D_frame_config.h`;
3. 2D frame structure functional configuration — `radar_framework_2D_frame_config.h`;
4. Interval0 range FFT acquisition configuration — `radar_framework_Interval0_report_config.h`;
5. Motion point cloud configuration — `radar_framework_motion_pointcloud_config.h`;
6. Presence point cloud configuration — `radar_framework_presence_pointcloud_config.h`;
7. HIF configuration — `radar_framework_hif_config.h`;
8. System interface configuration — `radar_framework_soc_config.h`.

4.2.2 Global radar framework Configuration

4.2.2.1 Globally Enabling the 1D Frame Structure

Use **CONFIG_RADAR_FRAMEWORK_1D_FRAME_EN** to globally enable the 1D frame structure. Only when this macro is enabled can the waveform be configured in 1D frame mode and all 1D frame structure code be compiled.

If the 1D frame structure is not used, disable it to save Static Random-Access Memory (SRAM). When this function is enabled and the waveform uses a 1D frame structure, multiple frames can be buffered and read by the Host in a single operation. This is useful when the Host read latency is uncertain.

4.2.2.2 Globally Enabling the 2D Frame Structure

Use **CONFIG_RADAR_FRAMEWORK_2D_FRAME_EN** to globally enable the 2D frame structure. Only when this macro is enabled can the waveform be configured in 2D frame mode and all 2D frame structure functions described in this section be compiled. If the 2D frame structure is not used, disable it to save SRAM.

4.2.2.3 Configuring the Maximum Supported Number of Range FFT Points

Use '**CONFIG_RADAR_FRAMEWORK_MAX_RFFT_LEN**' to configure the maximum number of range FFT points supported by radar framework. During static compilation, the maximum heap allocation must be determined from the number of range FFT points. Therefore, the maximum supported number of range FFT points must be specified in the global radar framework configuration.

During initialization, radar framework checks the configured number of range FFT points against the maximum supported number. If the configured number is greater, an error message is displayed and initialization fails.

Default value: 256.

4.2.2.4 Configuring the DataCube Buffer

On a Host running a non-real-time operating system, interrupt response may be delayed, causing reported DataCube data to be lost when the timing margin is small. To address this, radar framework provides buffered reporting of multiple DataCube frames. If a new frame arrives before the Host responds to the current frame's DataCube interrupt, the radar can buffer several frames. When the Host becomes available, it can read all buffered DataCubes together to maintain data continuity.

Use **CONFIG_RADAR_FRAMEWORK_DATACUBE_REPORT_SIZE** and **CONFIG_RADAR_FRAMEWORK_DATACUBE_LATENCY_REPORT_FRAME_CNT** to configure the maximum supported size of a single DataCube frame (Bytes) and the number of DataCube frames to buffer, respectively. This allows the HIF interface to allocate the required heap space appropriately.

Buffer as many frames as SRAM permits to reduce packet loss. For details of the DataCube buffering mechanism, refer to '[RS6x 7x HIF Reference Manual.pdf](#)'.

4.2.2.5 Configuring Adaptive Gain

Adaptive gain helps prevent echo overflow or echoes being buried in quantization noise when targets with very large and very small Radar Cross Sections (RCS) appear in the same spectrum.

In a 2D frame structure, adaptive gain can be enabled in hardware during range and Doppler FFT processing. The 1D frame structure does not support adaptive gain. Adaptive gain is not recommended for DataCube reporting applications because it reduces transmission efficiency and increases the Host processing load. For more information, refer to '[RS6x 7x mmWave Radar Point Cloud Processing Reference Manual.pdf](#)'.

Default configuration: disabled.

4.2.2.6 Reporting Frame Information

Use **CONFIG_RADAR_FRAMEWORK_REPORT_FRAME_INFO** to control reporting of frame parameters, heap usage, computation time, and other information through the PSIC debug protocol (MsgId = 0xC6) before each waveform transmission. Disable this macro in projects that report DataCube data to improve reporting efficiency. The PSIC debug protocol can be visualized in RadarDebugTool.

Default configuration: enabled.

4.2.2.7 Enabling Heartbeat Reporting

Use **CONFIG_RADAR_FRAMEWORK_REPORT_HEART_BEAT** to control heartbeat reporting. Heartbeats are typically used in scenarios with sleep and wake-up enabled. Therefore, only the Tracking project enables this macro by default. The default heartbeat interval is 10 min. For implementation details, refer to '[RS6x 7x mmWave Radar Development Quick Start Guide.pdf](#)'.

4.2.2.8 Enabling the Watchdog

Use **CONFIG_MMW_WATCHDOG** to enable the watchdog. When enabled, **CONFIG_WDG_TIMEOUT_MS** controls the watchdog reset timeout, which defaults to 5 seconds.

The watchdog is enabled in all projects by default. For implementation details, refer to '[RS6x 7x mmWave Radar Development Quick Start Guide.pdf](#)'.

4.2.2.9 Enabling Task Monitor

Use **CONFIG_MMW_LOOP_TASK_MONITOR** to enable callback monitoring. This function overlaps with the watchdog when sleep and wake-up are disabled. However, at certain sleep levels, the watchdog stops timing

after the system enters sleep. Task Monitor is therefore necessary as an auxiliary monitoring mechanism when sleep and wake-up are enabled.

Task Monitor is enabled in all projects by default. For implementation details, refer to '[RS6x 7x mmWave Radar Development Quick Start Guide.pdf](#)'.

4.2.3 1D Frame Structure Subfunction Configuration

The 1D frame structure supports the following configurations:

- Configure ADC data output;
- Report 1D-DataCube.

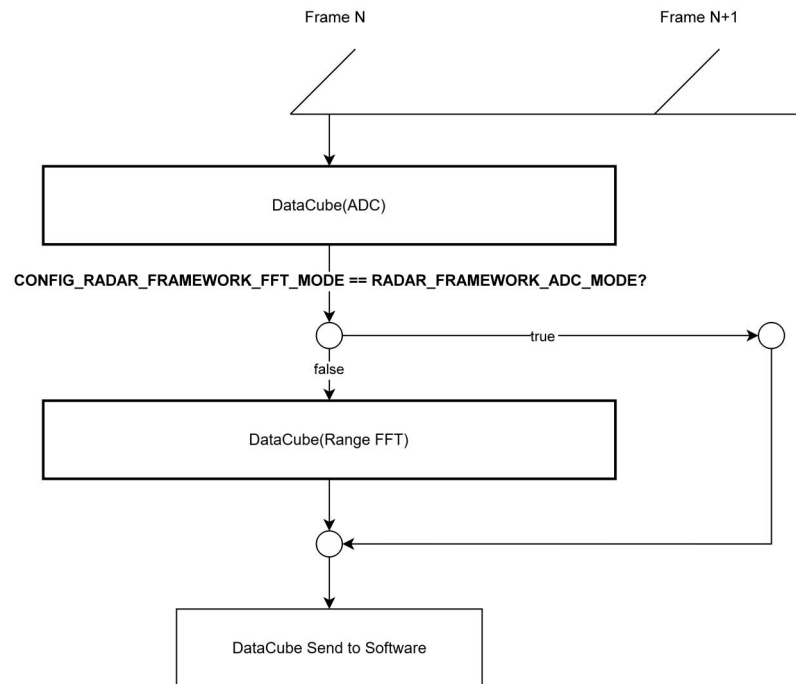


Figure 4-2 Data Path in the 1D Frame Structure

4.2.3.1 Reporting 1D-DataCube

Use **CONFIG_RADAR_FRAMEWORK_REPORT_1D_DATA_CUBE** to control 1D-DataCube reporting. Default value: 1 (1D-DataCube reporting enabled).

4.2.3.2 Heap Usage Configuration

HEAP_SIZE_1D_FRAME_CUBE indicates the SRAM size required for 1D-DataCube reporting. HIF automatically calculates the SRAM required to report 1D-DataCube.

If 1D-DataCube reporting is disabled, radar framework only buffers the DataCube and passes it to the user callback. SRAM usage is calculated as follows:

Number of range FFT points * Number of Rx antennas * Number of Tx antennas * Number of buffered frames * 4 Bytes

4.2.4 2D Frame Structure Functional Configuration

4.2.4.1 Globally Enabling the 2D Frame Structure

Use **CONFIG_RADAR_FRAMEWORK_2D_FRAME_EN** as the global switch for 2D frame structure functions. Only when this macro is enabled can the waveform be configured in 2D frame mode and all 2D frame structure functions described in this section be compiled. If the 2D frame structure is not used, disable this macro to save SRAM. Default value: 1.

In the 2D frame process, both 1D and 2D FFTs are performed by hardware. The supported process controls are shown below:

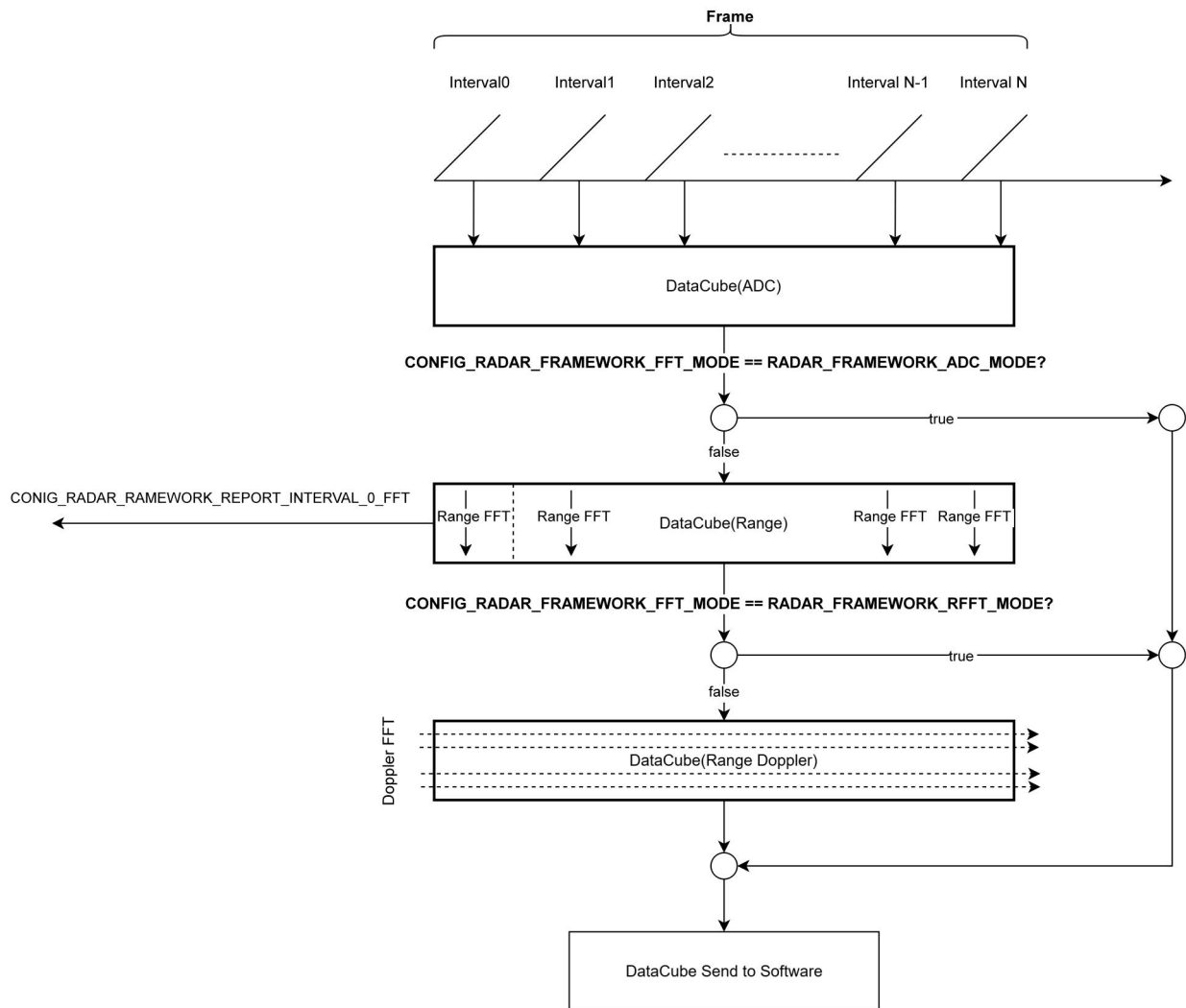


Figure 4-3 Data Path in the 2D Frame Structure

4.2.4.2 Reporting 2D-DataCube

Use **CONFIG_RADAR_FRAMEWORK_REPORT_2D_DATA_CUBE** to configure nonblocking reporting of 2D-DataCube with HIF message ID 0xC2 in the 2D frame callback. The typical time to report 256K of 2D-DataCube data through single-line SPI is 85 ms. Reporting 2D-DataCube through Universal Asynchronous Receiver/Transmitter (UART) is not recommended. For further HIF reporting limitations and implementation details, refer to ['RS6x 7x HIF Reference Manual.pdf'](#).

Default value: 0.

4.2.5 Obtaining the Range FFT of Interval0

Use **CONFIG_RADAR_FRAMEWORK_INTERVAL0_RFFT_EN** to control buffering of the range FFT data of the first Interval in 2D frame mode (INTERVAL0_RFFT). INTERVAL0_RFFT is generally used for breathing and heartbeat measurement in the slow-time dimension.

Default value: 0.

After enabling Interval0 range FFT data reporting, the following macros can be configured further.

- **Configuring INTERVAL0_RFFT Reporting**

Enable **CONFIG_RADAR_FRAMEWORK_REPORT_INTERVAL0_RFFT** to report Interval0 range FFT data with HIF message ID 0xC2 in a nonblocking manner in each callback.

- **Configuring Heap Space for INTERVAL0_RFFT**

Use **HEAP_SIZE_1D_CUBE** to configure the additional SRAM required to buffer INTERVAL0_RFFT. The radar buffers the range FFT data in a ping-pong buffer. The required SRAM is calculated as follows:

$$\text{Number of range FFT points} * \text{Number of Rx antennas} * \text{Number of Tx antennas} * 4 \text{ Bytes} * 2$$

4.2.6 Motion Point Cloud

Enabling **CONFIG_MMW_MOTION_POINT_CLOUD** enables motion point cloud computation and reserves heap space for it in the global heap. In each 2D frame callback, radar framework calls the interfaces provided by the point cloud computation subsystem ([subsys/mmwave/mmwave_application](#)) to calculate point cloud information. For details on how software calculates target positions, or to develop a custom point cloud algorithm using the mmwave_ctrl API, refer to ['RS6x 7x mmWave Radar Point Cloud Processing Reference Manual.pdf'](#).

After enabling motion point cloud computation, the following macros can be configured further.

4.2.6.1 Configuring Software CFAR

When **CONFIG_MMW_SW_CFAR_ENABLE** is enabled, radar framework uses software Constant False Alarm Rate (CFAR) processing instead of hardware CFAR results to calculate point clouds, providing faster computation but lower point cloud density. This macro is used by the modules in [subsys/mmwave/mmwave_application](#). Refer to ['RS6x 7x mmWave Radar Point Cloud Processing Reference Manual.pdf'](#).

Default value: 0.

Note: Adaptive gain must be disabled when software CFAR is enabled; otherwise, an error occurs during preprocessing.

4.2.6.2 Configuring Motion Point Cloud Reporting

When **CONFIG_RADAR_FRAMEWORK_MOTION_POINT_CLOUD_REPORT** is enabled, radar framework reports motion point clouds through HIF. Developers can disable this macro when using the motion point cloud directly for subsequent processing.

Default value: 1.

4.2.6.3 Configuring the Maximum Number of Motion Point Cloud CFAR Points

Use **CONFIG_MMW_MAX_MOTION_CFAR_NUM** to configure the maximum number of motion point cloud CFAR points. Reducing this number shortens computation time. If the number of detected motion point cloud points exceeds the configured value, distant targets are discarded.

Default value: 512.

4.2.6.4 Motion Point Cloud Heap Configuration

Use **HEAP_SIZE_MOTION** to configure the motion point cloud heap size. The required heap size depends on the configured maximum number of CFAR points. Motion point cloud heap usage is as follows:

Motion point cloud CFAR result storage: $512 * 4 \text{ (Bytes)} + \text{CONFIG_MMW_MAX_MOTION_CFAR_NUM} * 4 \text{ (Bytes)}$. Hardware CFAR produces up to 512 results, while the number of software CFAR results equals the maximum number of CFAR points;

Motion point cloud computation result storage: $\text{CONFIG_MMW_MAX_MOTION_CFAR_NUM} * 24 \text{ (Bytes)}$; motion point cloud reporting storage: $\text{CONFIG_MMW_MAX_MOTION_CFAR_NUM} * 12 \text{ (Bytes)}$, requiring motion point cloud reporting to be enabled;

Motion point cloud initialization space: $\text{CONFIG_RADAR_FRAMEWORK_MAX_RFFT_LEN} * 8$,

Total motion point cloud heap usage: Motion point cloud CFAR result storage + Motion point cloud computation result storage + Motion point cloud reporting storage +

$\text{CONFIG_MMW_MAX_MOTION_CFAR_NUM} * 1 \text{ (Bytes)}$ (heap required for low-speed filtering) + Motion point cloud initialization space.

4.2.7 Presence Point Cloud

Enabling **CONFIG_MMW_PRESENCE_POINT_CLOUD** enables **presence point cloud computation** and reserves heap space for it in the global heap. In each 2D frame callback, radar framework calls interfaces provided by [subsys/mmwave/mmwave_application](#) to calculate presence point cloud information.

The basic presence point cloud processing flow is as follows: Based on the configured division factor N, buffer velocity bin 0 from the DataCube every N frames to form a DataCube in the slow-time dimension. Perform a Doppler FFT in the slow-time dimension, and use CFAR to detect presence targets.

After enabling the presence point cloud function, the following macros can be configured further:

4.2.7.1 Configuring the Maximum Number of Presence Point Cloud Points

Use **CONFIG_MMW_PRESENCE_POINT_MAX** to configure the maximum number of detected presence point cloud points. Reducing this number shortens computation time but may discard distant targets.

Default value: 256.

4.2.7.2 Configuring the Presence Point Cloud Detection Method

Use **CONFIG_MMW_PRESENCE_POINT_CLOUD_PROCESS_METHOD** to configure the presence point cloud detection algorithm. The available algorithms are SNR threshold detection and CFAR detection. Define **CONFIG_MMW_PRESENCE_POINT_CLOUD_PROCESS_METHOD** as 0 to use SNR detection with a fixed threshold. This method provides good detection performance in relatively simple environments with concentrated targets. Compared with CFAR detection, it requires less computation and less SRAM.

Define **CONFIG_MMW_PRESENCE_POINT_CLOUD_PROCESS_METHOD** as 1 to use CFAR detection with a linear threshold. CFAR detection has two modes, selected by **CONFIG_MMW_MICRO_POINT_CLOUD_CFAR_MODE**. To reduce SRAM usage and computation, the presence point cloud algorithm supports single-sided noise-floor estimation for CFAR. This mode is suitable for open environments with sparse targets. Compared with double-sided CFAR, it causes almost no performance loss in such scenarios and saves 10 KB of SRAM.

Define **CONFIG_MMW_MICRO_POINT_CLOUD_CFAR_MODE** as 1 to select double-sided CFAR. It adaptively estimates the noise floor using samples on both sides, providing good noise immunity and multi-target detection performance. This mode is suitable for complex environments with relatively sparse targets.

Default: 1 (CFAR detection);

1 (double-sided CFAR).

4.2.7.3 Configuring the Presence Point Cloud Detection Threshold

Use **CONFIG_MMW_PRESENCE_POINT_CLOUD_CFAR_TH_DB** to configure the presence point cloud detection threshold. For SNR threshold detection, this value is the constant threshold. For CFAR detection, it is the minimum limit of the linear threshold. A higher value produces fewer noise points and lower target-point density, increasing the probability of missed detections. A lower value produces more target points and noise points, increasing the probability of false detections.

Use **CONFIG_MMW_PRESENCE_POINT_CLOUD_CFAR_LINEAR_TH_OFFSET_DB** to configure the overall offset of the presence point cloud detection threshold. For SNR threshold detection, this parameter has the same function as **CONFIG_MMW_PRESENCE_POINT_CLOUD_CFAR_TH_DB**. For CFAR detection, it raises the entire linear threshold.

Default: 8.5 (detection threshold);

1.5 (threshold offset).

Note: If the presence point cloud detection method is set to SNR threshold detection, changing the detection threshold to 10.5 is recommended.

4.2.7.4 Configuring the Number of Presence Point Cloud Range Bins

Use **CONFIG_MMW_PRESENCE_RANGE_BIN_NUM** to configure the farthest detection range-bin index for the presence point cloud. Presence point cloud detection capability is limited, and a larger detection area introduces more environmental interference. Long detection ranges are therefore generally unnecessary. This macro limits the detection range to application requirements, substantially reducing the required heap space and computation time.

Default value: 256 (only the first 256 bins are detected for the presence point cloud).

4.2.7.5 Configuring the Presence Point Cloud Range Decimation Factor

Use **CONFIG_MMW_PRESENCE_POINT_CLOUD_RANGE_DEC** to configure the presence point cloud range decimation factor. The default is 2, so only targets in range bins 0/2/4/6/8... are processed. This reduces computation and halves the storage required for the slow-time DataCube, but reduces target presence point cloud density.

Default value: 2 (range decimation factor of 2).

4.2.7.6 Configuring the Number of Buffered Presence Point Cloud Intervals

Use **CONFIG_MMW_PRESENCE_INTERVAL_NUM_LOG2** to configure the number of buffered presence point cloud intervals. Configure this macro as a LOG value. These intervals are used for an FFT in the slow-time dimension to obtain the presence point cloud spectrum.

Default value: 3 (8 buffered intervals).

4.2.7.7 Configuring the Number of Presence Point Cloud Doppler FFT Points

Use **CONFIG_MMW_PRESENCE_DFFT_LEN_LOG2** to configure the FFT length for the presence point cloud slow-time DataCube. Configure this macro as a LOG value. The number of Doppler FFT points must be greater than or equal to the number of buffered Intervals.

Default value: 4 (Doppler FFT length of 16).

4.2.7.8 Presence Point Cloud Heap Configuration

1. Use **HEAP_SIZE_PRESENCE** to configure the presence point cloud heap size. The required size depends on the presence point cloud CFAR mode and decimation factor. Presence point cloud heap usage is as follows:
2. Presence point cloud computation result storage: **CONFIG_MMW_PRESENCE_POINT_MAX** * 24 (Bytes);
3. Calculate the buffered Interval count and Doppler FFT length: $1 \ll \text{CONFIG_MMW_PRESENCE_INTERVAL_NUM_LOG2}$ and $1 \ll \text{CONFIG_MMW_PRESENCE_DFFT_LEN_LOG2}$;
4. Actual number of presence point cloud range bins: **CONFIG_MMW_PRESENCE_RANGE_BIN_NUM** / **CONFIG_MMW_PRESENCE_POINT_CLOUD_RANGE_DEC** + 1;
presence point cloud Cube size: Actual number of presence point cloud range bins * Rx * Tx * (Number of Intervals + 1) * 4 (Bytes);
5. Presence point cloud Interval buffer space: Rx * Tx * (Number of Intervals + 1) * 4 (Bytes);
6. Calculate the space unit required for the maximum window length. The length depends on whether **CONFIG_MMW_MICRO_POINT_CLOUD_CFAR_MODE** specifies single-sided or double-sided CFAR:
 - Single-sided CFAR: $(37 / \text{CONFIG_MMW_PRESENCE_POINT_CLOUD_RANGE_DEC} + 1) * 4$ (Bytes);
 - Double-sided CFAR: $(73 / \text{CONFIG_MMW_PRESENCE_POINT_CLOUD_RANGE_DEC} + 1) * 4$ (Bytes);
7. Presence point cloud computation buffer space:
 - SNR threshold method: $4 * 3 * \text{number of Doppler FFT points} * \text{Rx} * \text{Tx}$;
 - CFAR detection: $(\text{number of Doppler FFT points} * (\text{Rx} * \text{Tx}) * \text{window-length space units}) + ((2 * \text{number of Doppler FFT points} + 1) * (\text{number of range bins actually used for the presence point cloud} * 4))$;
8. Presence point cloud reporting storage: **CONFIG_MMW_PRESENCE_POINT_MAX** * 12 (Bytes);
total presence point cloud heap usage: Presence point cloud computation result storage + Presence point cloud slow-time DataCube storage + Presence point cloud Interval buffer space + Presence point cloud FFT space + Presence point cloud reporting storage.

4.3 Application Configuration Based on radar framework

The SDK provides several Samples based on radar framework. This section describes how their mmWave radar waveforms and point cloud performance are configured for different applications, and how to tailor functions and performance.

4.3.1 Waveform Configuration

Waveform parameters and key timing definitions are shown below. For more information on waveform configuration, refer to ['RS6x 7x mmWave Radar Reference Manual.pdf'](#).

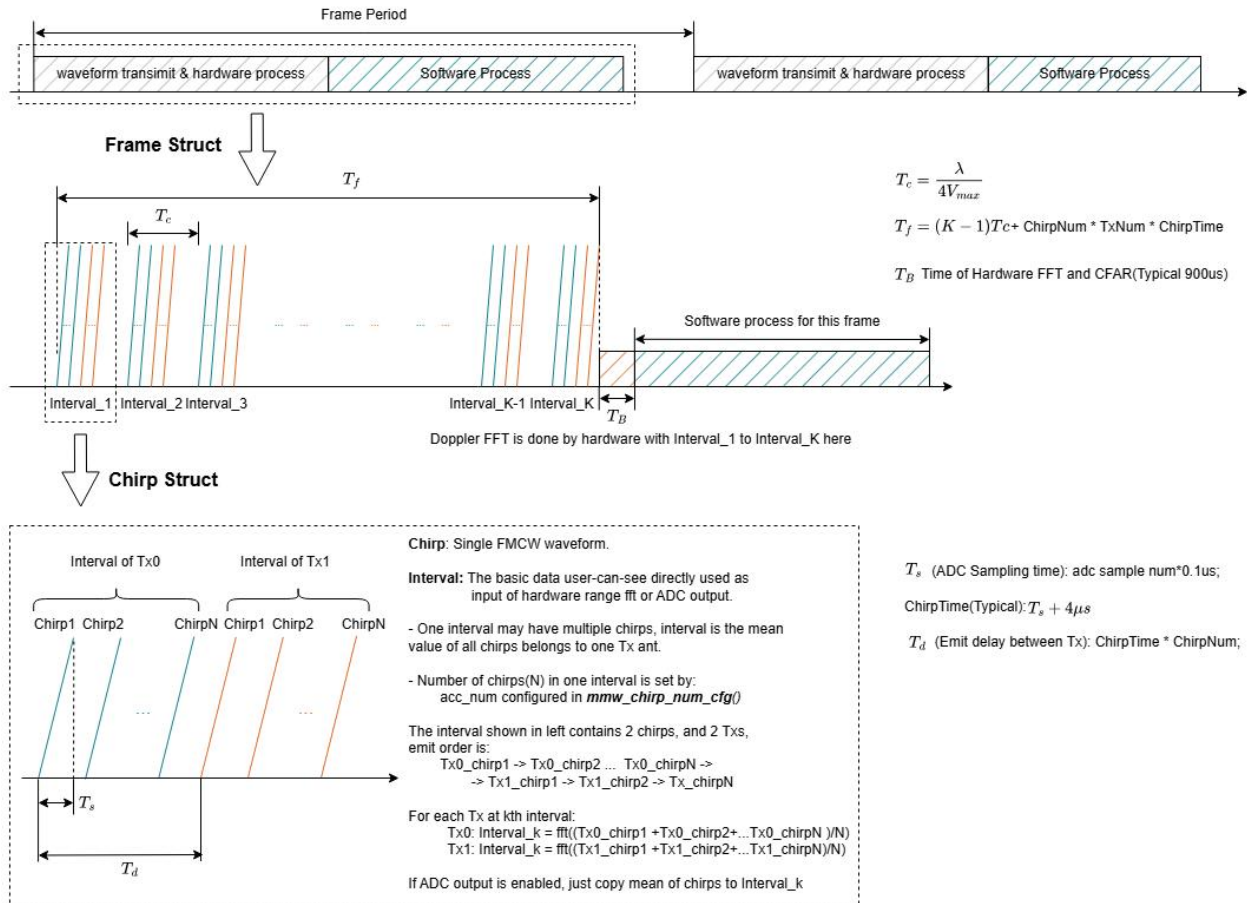


Figure 4-4 Waveform Structure

Waveform configuration includes the following configurable items.

4.3.1.1 MIMO Mode

Use **RADAR_FRAMEWORK_SAMPLE_MIMO_MODE** to configure the antenna operating mode for Multiple-Input Multiple-Output (MIMO). For 6130 series chips, only MMW_MIMO_1T3R is supported. For 6240 series chips, only MMW_MIMO_2T4R is supported.

4.3.1.2 Start Frequency

Use **RADAR_FRAMEWORK_SAMPLE_START_FREQ_MHZ** to configure the start frequency. RS6240 series chips support the full frequency range of 57 GHz–66 GHz, while RS6130 series chips support 58 GHz–64 GHz.

4.3.1.3 Frame Type

Use **RADAR_FRAMEWORK_SAMPLE_FRAME_TYPE** to configure the default frame structure to transmit. A frame structure can be selected only after **CONFIG_RADAR_FRAMEWORK_1D_FRAME_EN** or **CONFIG_RADAR_FRAMEWORK_2D_FRAME_EN** is enabled; otherwise, an error is reported during frame configuration. For a 1D frame structure, Interval_K in Figure 4-4 is always 1, and data processing follows Figure 4-2.

4.3.1.4 Number of Range FFT Points (log2)

Use **RADAR_FRAMEWORK_SAMPLE_RFFT_LEN_LOG2** to configure the number of range FFT points in log2 form. The maximum value is 10 with adaptive gain disabled, or 9 with adaptive gain enabled. For example, if **RADAR_FRAMEWORK_SAMPLE_RFFT_LEN_LOG2** is 8, the number of range FFT points and ADC samples is $2^8 = 256$.

Configuration note 1: A range FFT point count (log2) of 7/8/9 is generally recommended for a good balance between performance and power consumption. radar framework only supports static configuration of a power-of-two range FFT length, which is suitable for most scenarios. Too many range FFT points greatly increase computation and storage requirements; too few reduce detection capability.

Configuration note 2: The number of range FFT points must be less than the number represented by **CONFIG_RADAR_FRAMEWORK_MAX_RFFT_LEN**;

Configuration note 3: If **RADAR_FRAMEWORK_SAMPLE_RFFT_LEN_LOG2** is **UINT16_MAX**, radar framework does not use `range_fft_len_log2` as the FFT point count. Instead, it calculates the ADC sample count from

max_range_mm / range_resolution_mm in RadarConfig_t and selects a power of two no smaller than the ADC sample count as the range FFT length. The number of range FFT points is then:

$$\text{power}(2, \text{ceil}(\log_2(\text{max_range_mm}/\text{range_resolution_mm})));$$

Configuration note 4: If range FFT processing is disabled, only a power-of-two range FFT point count can be configured.

4.3.1.5 Range Resolution

Use **RADAR_FRAMEWORK_SAMPLE_RANGE_RESOLUTION_MM** to configure range resolution. A smaller range resolution produces denser point clouds and more precise range measurements. Range resolution and the number of range FFT points jointly determine the maximum detection range:

$$\text{Maximum detection range} = (1 \ll \text{RADAR_FRAMEWORK_SAMPLE_RFFT_LEN_LOG2}) * \text{RADAR_FRAMEWORK_SAMPLE_RANGE_RESOLUTION_MM};$$

4.3.1.6 Number of Doppler FFT Points (log2)

Use **RADAR_FRAMEWORK_SAMPLE_DFFT_LEN_LOG2** to configure the number of Doppler FFT points in log2 form, with a maximum value of 10. For example, if **RADAR_FRAMEWORK_SAMPLE_DFFT_LEN_LOG2** is 8, the number of Doppler FFT points and Intervals is $2^8 = 256$.

Note: The maximum DataCube size is 256 KB (RS624x) or 192 KB (RS6130). The firmware includes a compression algorithm supporting up to one-fold compression, so the maximum supported DataCube sizes are 512K and 384K, respectively. DataCube storage is calculated as follows:

$$\text{Number of range FFT points} * \text{Number of Doppler FFT points} * \text{Number of Tx antennas} * \text{Number of Rx antennas} * 4 \text{ Bytes (IQ)}$$

To meet the above requirements,

$$\text{RADAR_FRAMEWORK_SAMPLE_DFFT_LEN_LOG2} + \text{RADAR_FRAMEWORK_SAMPLE_RFFT_LEN_LOG2} \leq 14;$$

4.3.1.7 Velocity Resolution

Use **RADAR_FRAMEWORK_SAMPLE_VEL_RESOLUTION_MM** to configure velocity resolution. The velocity resolution and number of Doppler FFT points determine the radar's effective velocity measurement range:

$$[-\text{Velocity resolution} * \text{Number of Doppler FFT points}/2, \text{Velocity resolution} * \text{Number of Doppler FFT points}/2]$$

Velocity resolution affects the waveform transmission duration of a Frame. The Frame transmission time can be quickly estimated using the following formula:

$$T_f = \frac{\lambda}{2V_{res}}$$

4.3.1.8 Number of Accumulated Chirps (log2)

Use **RADAR_FRAMEWORK_SAMPLE_ACC_NUM_LOG2** to configure the number of accumulated Chirps in an Interval. The radar averages all Chirps in the Interval in the complex domain and uses the result as the Interval sample for the range FFT. The number of Chirps per Interval is $(1 \ll \text{RADAR_FRAMEWORK_SAMPLE_ACC_NUM_LOG2})$. Chirp accumulation can improve detection capability. However, if the unambiguous velocity is too small, all Chirps may not fit between two Intervals, resulting in abnormal radar waveform transmission.

RADAR_FRAMEWORK_SAMPLE_ACC_NUM_LOG2 should be set to 3 or less. A value of 0 means that only one Chirp is transmitted per Interval.

4.3.1.9 FFT Mode

Set **CONFIG_RADAR_FRAMEWORK_FFT_MODE** to **RADAR_FRAMEWORK_ADC_MODE**/**RADAR_FRAMEWORK_RFFT_MODE**/**RADAR_FRAMEWORK_2DFFT_MODE** to select no range FFT, range FFT only, or the full 2D-FFT process in Figure 4-3, respectively.

If **CONFIG_RADAR_FRAMEWORK_FFT_MODE** is **RADAR_FRAMEWORK_ADC_MODE**, the radar performs neither range nor Doppler FFTs. The reported DataCube contains the ADC samples of each Interval instead.

- **Limitation 1:** In **RADAR_FRAMEWORK_ADC_MODE**, motion and presence point clouds cannot be calculated, and a message is printed during app initialization;
- **Limitation 2:** In **RADAR_FRAMEWORK_ADC_MODE**, the maximum detection range must be a power-of-two multiple of the range resolution, and the maximum measurable velocity must be a power-of-two multiple of the velocity resolution. Otherwise, data corruption may occur.

If **CONFIG_RADAR_FRAMEWORK_FFT_MODE** is **RADAR_FRAMEWORK_RFFT_MODE**, the radar does not perform the Doppler FFT. The reported DataCube contains the range FFT data of each Interval in the 2D frame.

- **Limitation 1:** In **RADAR_FRAMEWORK_RFFT_MODE**, motion and presence point clouds cannot be calculated, and a message is printed during app initialization;
- **Limitation 2:** In **RADAR_FRAMEWORK_RFFT_MODE**, the maximum measurable velocity must be a power-of-two multiple of the velocity resolution. Otherwise, data corruption may occur.

If **CONFIG_RADAR_FRAMEWORK_FFT_MODE** is **RADAR_FRAMEWORK_2DFFT_MODE**, both range and Doppler FFTs are performed for a 2D frame structure, while only the range FFT is performed for a 1D frame structure.

4.3.1.10 Static Clutter Convergence Mode

Modify **RADAR_FRAMEWORK_SAMPLE_CLUTTER_HALT_METHOD** to configure the static clutter convergence mode (effective only when static clutter removal is enabled). In static clutter suppression, the radar hardware estimates echoes from stationary targets in the environment, referred to as static clutter, and subtracts them before the range FFT of each Interval. This suppresses stationary-target energy in the DataCube so that CFAR does not detect these targets.

Static clutter supports:

- Continuous update;
- No update after convergence;
- Automatic update;

Three update modes.

- **Continuous update:** The radar continuously estimates and tracks static clutter.

This mode is commonly used when the environment may change. It continuously tracks stationary-target energy, preventing persistent false detections when new targets appear or existing targets disappear. However, if a target of interest remains stationary for some time, it is suppressed as part of the environment and can no longer be measured;

- **No update after convergence:** The radar stops estimating static clutter after the first convergence. If static clutter removal is enabled, this fixed static clutter estimate is subtracted during each update.

This mode detects any difference from the initial environment. However, moving the radar or changes to objects that are not intended targets may cause persistent false detections. Ambient temperature changes can also invalidate the initial converged static clutter estimate. Therefore, when using this mode, the application layer must actively update static clutter periodically to ensure stability.

- **Automatic update:** Effective only when motion point cloud processing is enabled.

If no motion point cloud points are detected in the current frame, static clutter updating is enabled for the next frame; otherwise, updating is paused.

Note: This mechanism controls static clutter updating for the next frame based on whether the current frame contains point cloud points. Therefore, do not set the static clutter update time in the point cloud computation submodule too low; a value of 4 or greater is recommended. Also, if the radar Signal-to-Noise Ratio (SNR) setting is too low and persistent false detections occur, static clutter will not be updated.

4.3.2 Point Cloud Performance Configuration Guide

Performance configuration mainly includes waveform configuration and point cloud computation submodule configuration.

Waveform configuration is controlled by radar framework directly using the relevant macros in `prj_config.h`. Performance is configured by overriding the macros in [subsyst/mmwave_application/mmwave_pointcloud_config.h](#) in `prj_config.h`.

For more configuration options for the point cloud computation submodule, refer to ['RS6x 7x mmWave Radar Point Cloud Processing Reference Manual.pdf'](#).

4.3.3 Function Tailoring Guide

When developing with radar Samples, developers can disable unnecessary functions to reduce SRAM usage and leave more SRAM for their own applications.

The following methods can also be used to reduce features.

1. If the motion point cloud filtering method provided by the radar Sample is used and SRAM is limited, reduce **CONFIG_MMW_MAX_MOTION_CFAR_NUM** while ensuring that valid targets can still be detected;
2. If presence point cloud processing is used and SRAM is limited, reduce the maximum number of presence point cloud CFAR points or configure a larger range decimation factor while maintaining performance. For configuration details, see Section 4.2.7.

4.3.4 Typical Waveform Configurations

Using 6240 series chips as an example, typical waveform configurations for the five radar Samples are shown below:

Project Name	ReportDataCube1D	ReportDataCube2D	Posture	Sleep	Tracking
MIMO Mode	MMW_MIMO_2T4R	MMW_MIMO_2T4R	MMW_MIMO_2T4R	MMW_MIMO_2T4R	MMW_MIMO_2T4R
Start Frequency	57GHz	57GHz	57GHz	58.88GHz	58.88GHz
Frame Type	1DFFT	2DFFT	2DFFT	2DFFT	2DFFT
Number of Range FFT Points	256(2 ⁸)	256(2 ⁸)	256(2 ⁸)	256(2 ⁸)	256(2 ⁸)
Range Resolution	0.08m	0.08m	0.025m	0.04m	0.08m
Number of Doppler FFT Points	32(2 ⁵)	32(2 ⁵)	32(2 ⁵)	32(2 ⁵)	32(2 ⁵)
Velocity Resolution	0.25m/s	0.25m/s	0.1m/s	0.2m/s	0.2m/s
Number of Accumulated Chirps	1(2 ⁰)	1(2 ⁰)	1(2 ⁰)	1(2 ⁰)	4(2 ²)
Static Clutter Convergence Mode	Disabled	Disabled	Disabled	Disabled	Continuous update

4.3.5 Typical Performance Metrics

The following table shows the execution time per invocation of key modules, the computation time for 512 valid motion point cloud points, and the presence point cloud computation time per frame:

Table 4-1 Typical Computation Time

Computation time for 512 valid motion point cloud points	21.3ms
Presence Point Cloud SNR Threshold Detection Time	17 ms/frame

Presence Point Cloud CFAR Detection Time	13 ms/frame
--	-------------

The remaining SRAM for each Sample is shown below (cdk_6240_cpuf):

Table 4-2 Available SRAM

Project Name	Remaining SRAM / KB
Posture	89
Sleep	74
Tracking	150

4.4 Digital Signal Processing Framework

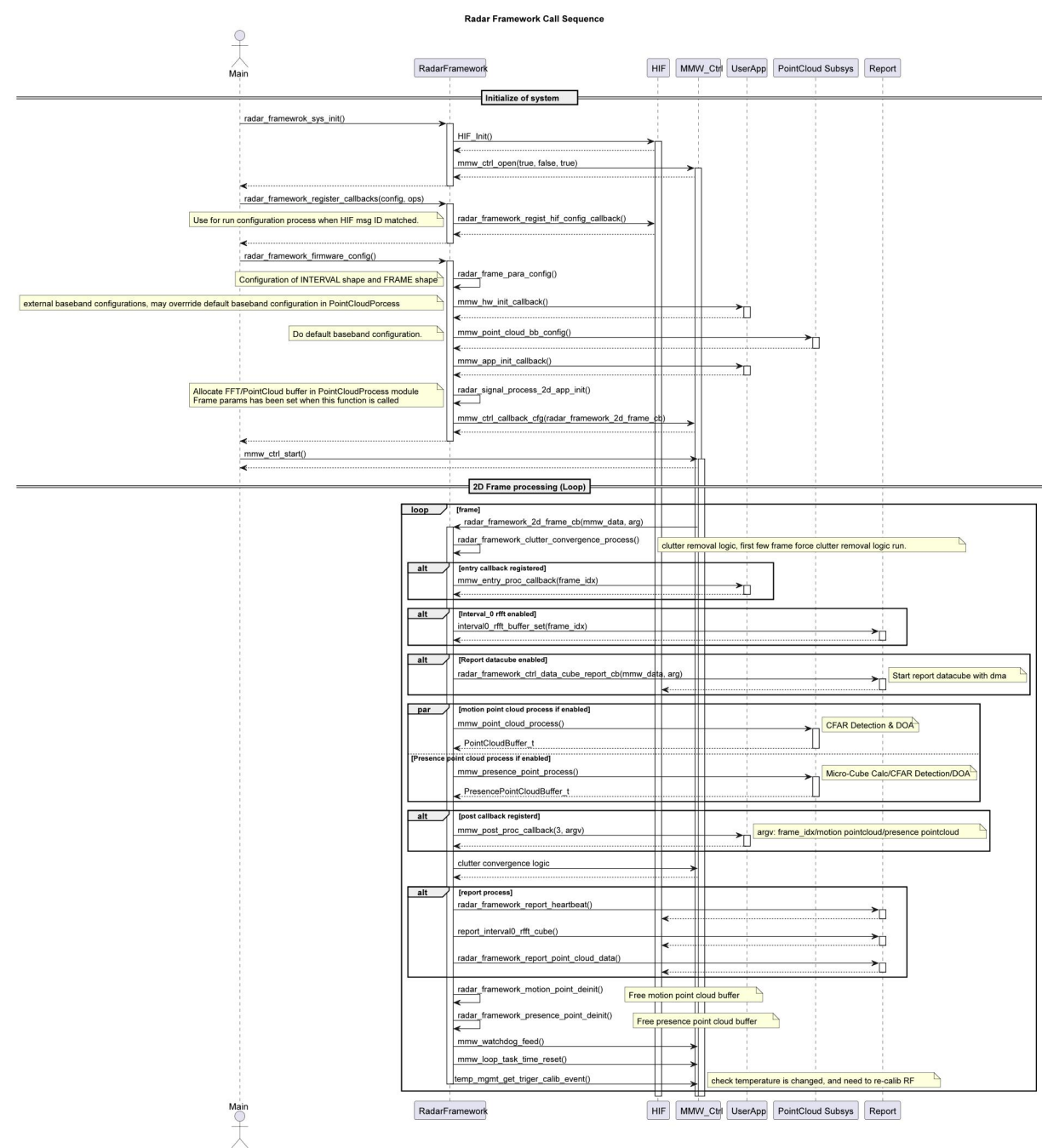


Figure 4-5 2D Digital Signal Processing Framework

4.5 HIF Configuration Framework

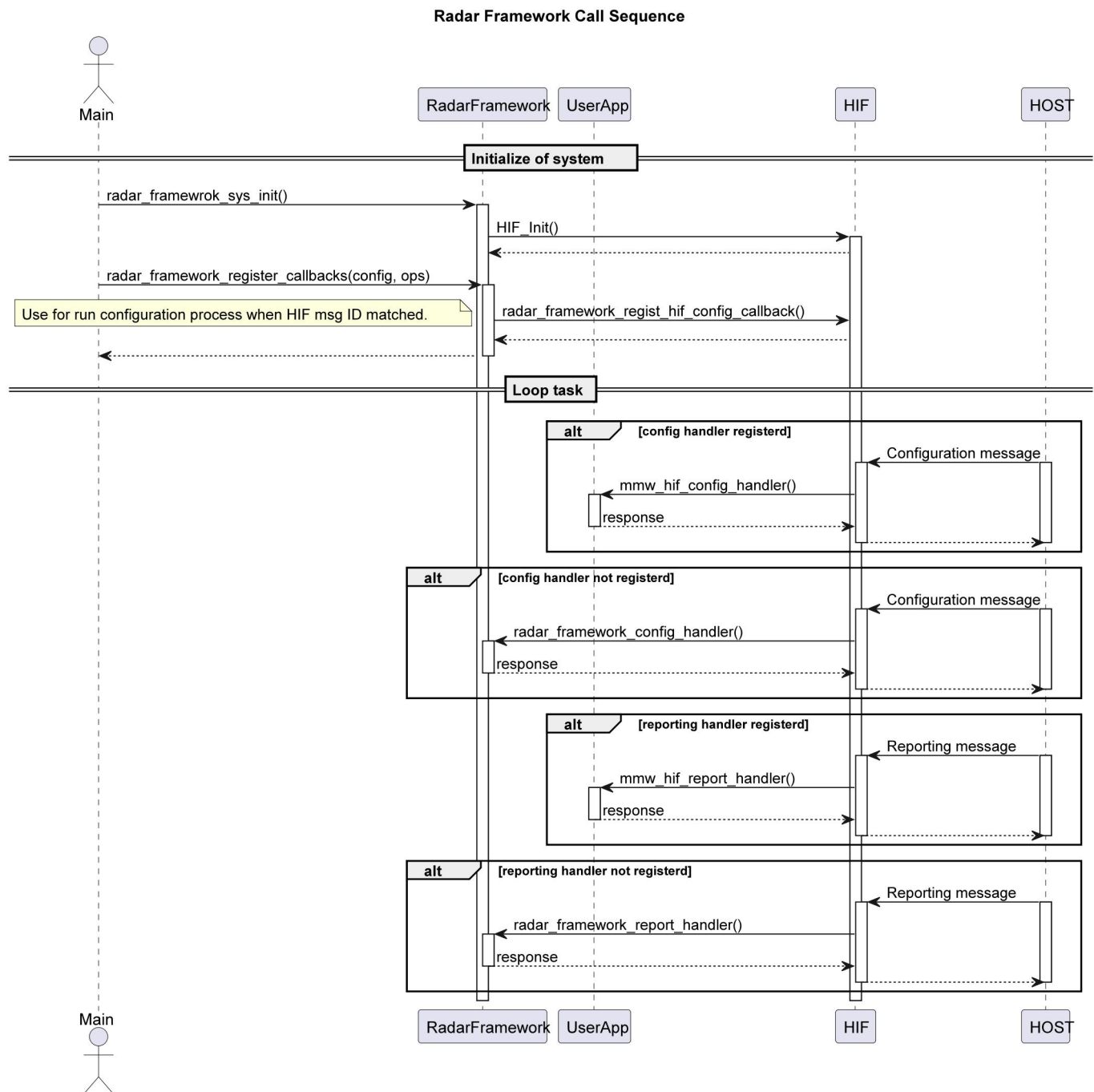


Figure 4-6 HIF Configuration Framework Call Flow

5 [Q & A]

5.1 How Can Point Cloud Density Be Increased?

Point cloud density is determined by the CFAR threshold, range resolution, number of range FFT points, velocity resolution, and number of Doppler FFT points. Smaller CFAR thresholds, range resolutions, and velocity resolutions increase density. Larger numbers of range and Doppler FFT points also increase density. However, increasing density also increases unwanted points.

5.2 How Can More SRAM Be Made Available?

Developers can reduce the space required by the code as follows:

- Set '**CONFIG_RADAR_FRAMEWORK_MAX_RFFT_LEN**' to the maximum supported number of range FFT points (log2);
- Disable unnecessary functions in `prj_config.h` to release the additional heap memory allocated to them;
- Refer to the **HEAP_SIZE_PRESENCE** calculation. SRAM usage can be reduced by reducing the number of presence point cloud range bins, the number of range decimation points, and the number of buffered intervals.

5.3 Why Does Point Cloud Output Stutter, and How Can Computation Time Be Reduced?

If the next frame finishes transmission and generates an interrupt before the current frame's computation is complete, the next frame's interrupt is not serviced. Its data is therefore not processed, causing point cloud output to stutter. The following methods can improve this:

- For motion point cloud applications, set **MMW_MAX_MOTION_CFAR_NUM** to a smaller value to limit the number of points and reduce computation time. In the worst case, outputting one motion point cloud point takes approximately 64 μ s;
- Increase the presence point cloud range decimation factor **CONFIG_RADAR_FRAMEWORK_PRESENCE_POINT_CLOUD_RANGE_DECM** to reduce the number of FFTs used to calculate the micro-motion spectrum and shorten computation time;
- Reduce **RADAR_FRAMEWORK_SAMPLE_RFFT_LEN_LOG2** or **RADAR_FRAMEWORK_SAMPLE_DFFT_LEN_LOG2** to reduce the number of range or Doppler FFT points. Halving either number approximately halves motion point cloud computation time and also reduces presence point cloud computation time. However, point cloud density also decreases;
- Configure appropriate range and velocity filters in the point cloud computation submodule ([subsys/mmwave/mmwave_application/mmwave_app_pointcloud_config.c](#)) to avoid computing unnecessary points and reduce computation time. For instructions, refer to ['RS6x 7x mmWave Radar Point Cloud Processing Reference Manual.pdf'](#);
- Increase the SNR threshold (`threshold_snr_pwr_lin`) in the point cloud computation submodule. Increasing the SNR requirement reduces the number of valid point cloud points and removes unwanted points, reducing computation time. For instructions, refer to ['RS6x 7x mmWave Radar Point Cloud Processing Reference Manual.pdf'](#);

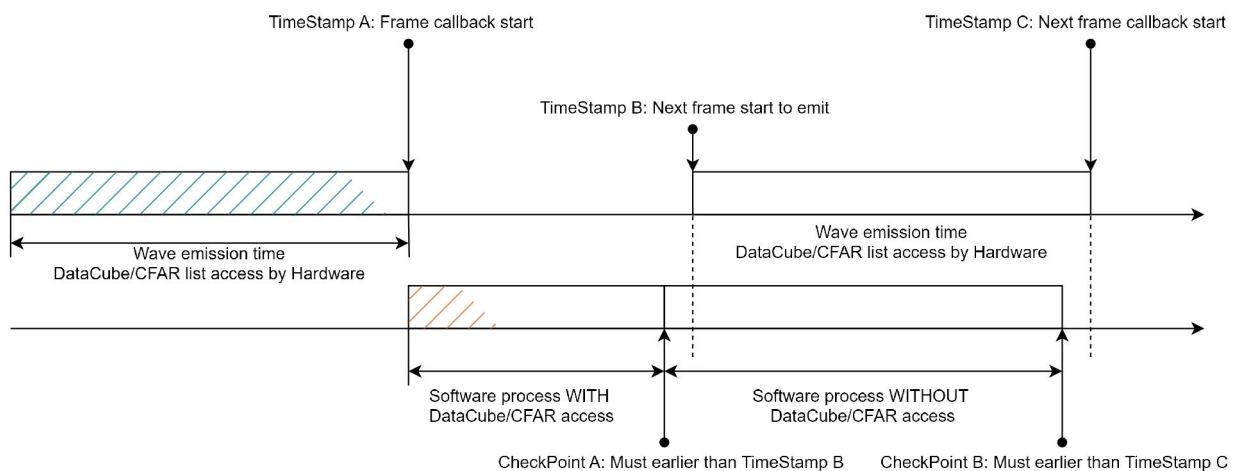
5.4 Are There Best Practices for Waveform Configuration?

- Range-Related Configuration
 - Unless there are special requirements, or if you are unfamiliar with radar digital signal processing, use **RADAR_FRAMEWORK_SAMPLE_RFFT_LEN_LOG2** to configure the number of range FFT points wherever possible to ensure that no zero-padding is used in range spectrum calculation;
 - If the appropriate range FFT length is unclear, use 256 or 512 points. If the computation load is too high, prefer 128 or 256 points; otherwise, prefer 256 or 512;
 - If the desired maximum detection range is x :

- ◆ Determine the range FFT point count using the above principles. The maximum detection range equals Range resolution * Number of range FFT points;
- ◆ Adjust the range resolution so that the maximum detection range is between $2x/3$ and $x/0.8$. This makes relatively full use of the spectrum while avoiding the last 20% of frequencies in the filter transition band;
- Velocity-Related Configuration
 - Determine the maximum velocity to measure. For pedestrians, the maximum measurable velocity is generally 3.2 m/s, giving a measurement range of -3.2 m/s to 3.2 m/s;
 - A Doppler FFT length of 32 is generally recommended. If the computation load is too high, use 16;
 - Calculate velocity resolution from the maximum velocity and Doppler FFT length: $2 * \text{Maximum measurable velocity} / \text{Number of Doppler FFT points}$;

5.5 What Are the Computation Timing Constraints?

radar framework checks whether the current frame's data meets the constraints at two checkpoints:



- (1) CheckPoint A: When DataCube access is complete (usually after motion point cloud computation and micro-motion point cloud buffering), radar framework checks whether transmission of the next frame has started. If so, a DataCube access conflict may already have occurred between software and hardware, making the current frame's data unreliable. radar framework stops further computation and waits for the next callback;
- (2) CheckPoint B: If the next frame finishes waveform transmission and generates an interrupt before processing of the current frame is complete, the next frame's interrupt is not serviced and its data is not processed.

5.6 How Can Detection of Weakly Reflecting Targets in the Radar Field of View Be Improved?

- (1) First, increase the number of accumulated Chirps using **RADAR_FRAMEWORK_SAMPLE_ACC_NUM_LOG2** to improve detection of weakly reflecting targets. This improves detection capability with almost no change to the original waveform parameters or point cloud density;
- (2) Second, increase **RADAR_FRAMEWORK_SAMPLE_RFFT_LEN_LOG2** and **RADAR_FRAMEWORK_SAMPLE_DFFT_LEN_LOG2** to improve weak-target detection. However, these changes affect point cloud density and computation requirements, so the upper-layer algorithm may need corresponding adjustments;
- (3) Finally, lower the CFAR threshold and perform additional point cloud filtering in the application layer to improve detection of weakly reflecting targets. For CFAR threshold adjustment, refer to ['RS6x 7x mmWave Radar Point Cloud Processing Reference Manual.pdf'](#);

Increasing **RADAR_FRAMEWORK_SAMPLE_ACC_NUM_LOG2** by 1 is theoretically equivalent, in detection performance, to doubling the number of range or Doppler FFT points.

5.7 Why Are Presence Point Cloud False Detections Likely at Short Range?

The radar radiates some energy backward, so a seated target behind the radar with a strong return may also be detected. Presence point cloud detection is highly sensitive and may detect people sitting behind the radar. Test personnel should therefore stay as far away from the radar as possible. If false detections persist, adjust the presence point cloud threshold in the SDK. Refer to ['RS6x 7x mmWave Radar Point Cloud Processing Reference Manual.pdf'](#).

5.8 Why Are There Many False Targets in Indoor Motion Point Clouds?

In confined environments containing many metal objects, the radar may produce many false targets. These mainly originate from two sources:

1. Targets are detected through multiple propagation paths. Typical multipath interference is shown below:

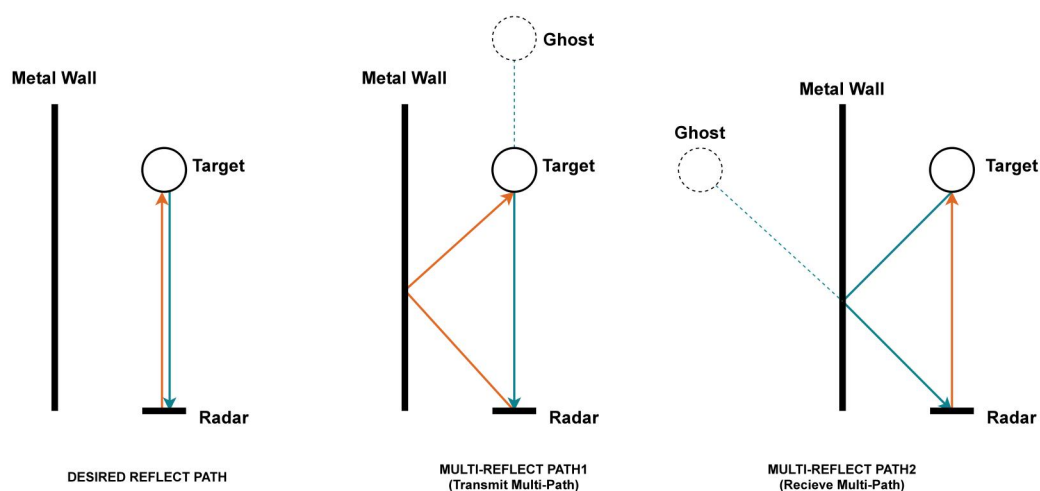


Figure 5-1 Multipath Interference

2. Moving targets occlude stationary targets behind them, but the occlusion varies. The energy of a stationary target is "amplitude-modulated" by a person's movement frequency and enters the velocity dimension, causing the stationary target to be identified as moving. If the radar is considered a light source, these stationary targets are detected as the person's "shadows".

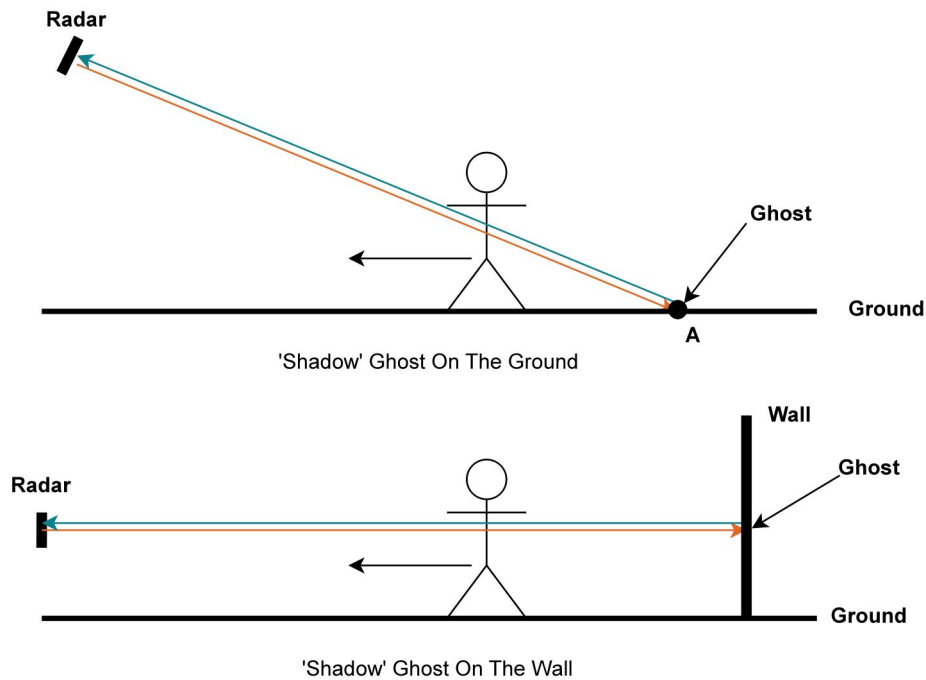


Figure 5-2 "Shadow" False Targets

Developers must remove these false targets during subsequent processing using energy, SNR, motion patterns, and other characteristics.

5.9 Why Might Configuration with RadarDebugTool Fail?

The Tracking/Posture/Sleep firmware in the SDK release supports at most 256 range FFT points. Configuring more than 256 points therefore results in a configuration failure message. To change the maximum supported number, set `CONFIG_RADAR_FRAMEWORK_MAX_RFFT_LEN` to 512 and recompile the firmware.

5.10 Why Are Spectra and Point Clouds Abnormal with Certain Chirp Accumulation Counts?

If ACC is too large, the configured number of Chirps cannot fit between Intervals, causing abnormal waveform transmission. The SDK currently cannot detect this condition. If it occurs, reduce ACC and check whether the abnormality disappears.

5.11 What Should Be Done When the Debug UART Prints malloc fail?

Memory allocation fails because insufficient heap space has been allocated. Increase `CONFIG_HEAP_SIZE`, which controls the heap size in `radar_framework_default_config.h`, to provide the required memory.

5.12 What Do the Error Messages Printed by the Debug UART Mean?

"The HIF task of the previous frame was not completed!": The computation timing violates CheckPoint B in Section 5.5. Check whether the Host responds to the HIF interrupt promptly and sends a Poll command to request HIF data.

"[RADAR_SIGNAL_2D_PROCESS] Calculate TIMEOUT!": The computation timing violates CheckPoint A in Section 5.5, indicating too many point cloud points. If this occurs continuously, increase the frame period or reduce `CONFIG_MMW_MAX_MOTION_CFAR_NUM` to reduce the computation load.

6 Appendix

6.1 Digital Signal Processing with ADC and 1D-DataCube Data

This section provides basic guidance for developers implementing point cloud algorithms using ADC or 1D-DataCube data.

6.1.1 Velocity Compensation Coefficient

For a moving target, the transmission interval between different transmit antennas creates a phase difference, causing angle measurement errors. Velocity compensation must therefore be applied to the transmit antennas before angle estimation. When 2D-FFT data is used, hardware has already performed velocity compensation. With ADC or 1D-DataCube data, developers must implement software velocity compensation.

For RS624X, the angle calibration formula for Tx1 can be expressed as:

$$\text{deta_phi} = (-1) * 2 * \pi * \text{dop_idx_shift} * \text{deta_phi_coef}$$

Here, `dop_idx_shift` is the velocity index after Doppler FFT shift, and `deta_phi_coef` is the velocity compensation coefficient, determined by velocity resolution and the ACC count.

$$\text{ant_vel_calib_data}[4 \sim 7] = \text{ant_data}[4 \sim 7] * (\cos(\text{deta_phi}) + j * \sin(\text{deta_phi}))$$

`ant_data` and `ant_vel_calib_data` are the data before and after velocity compensation, respectively. Compensation is applied only to Tx1 data.

Developers can obtain the velocity compensation coefficient using `mmw_get_dec_fcw()` and apply velocity compensation before angle estimation.

6.1.2 Angle Calibration Values

When developing signal processing with ADC or 1D-DataCube data, check whether hardware angle calibration has been performed in the time domain. For details, refer to ['RS6x 7x mmWave Radar Reference Manual.pdf'](#). If angle calibration has not been performed, apply software calibration before angle estimation. The angle calibration matrix can be obtained using `mmw_ant_calib_data_get()`.

6.1.3 ADC Data Point Cloud Processing Flow

Using ADC data from RS624X series modules as an example, the point cloud processing flow is shown in Figure 6-1 and mainly consists of the following steps:

1. Obtain the velocity compensation coefficient `deta_phi_coef` and antenna calibration array `ant_calib_arr`:
 - (1) `deta_phi_coef`: A U32Q20 value determined by the radar frame configuration;
 - (2) `ant_calib_arr`: An array of length 8 containing S16Q15 IQ values. The antenna calibration array may differ between modules and does not change with the radar configuration;
2. Perform FFT processing to convert ADC data into a DataCube. A Hann window is recommended during FFT processing. An FFT shift is required after the FFT;
3. Use CFAR to select targets and convert the DataCube into target range and velocity information;
4. Apply velocity compensation using the velocity compensation coefficient to compensate for the phase difference between tx0 and tx1 caused by target motion;
5. Use the calibration array for antenna calibration to correct angle measurement errors caused by antenna errors;
6. Estimate the angle using data after velocity compensation and antenna calibration to obtain target angle information.

For CFAR and angle estimation procedures, refer to `mmw_cfar_detection_sw_post_process()` and `mmw_process_angle_process()`

. Limitation 1: Velocity compensation in step 4 compensates for the phase difference between tx0 and tx1. RS613X series chips have only one transmit antenna, so this step is unnecessary;

Limitation 2: Hardware calibration is performed by default for RS613X series chips, so software angle calibration is unnecessary.

Limitation 3: Hardware angle calibration cannot be used with RS624X series chips.

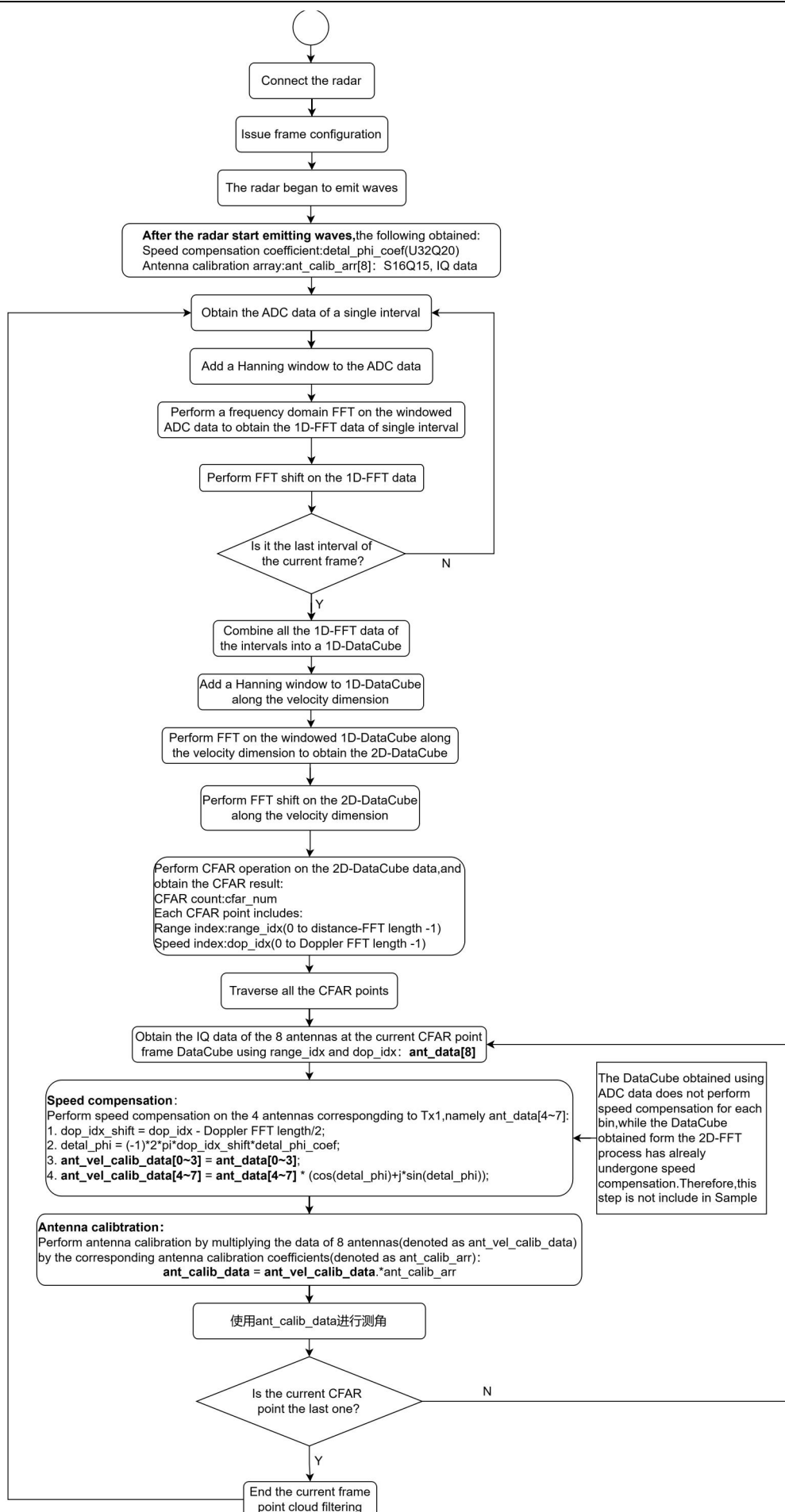


Figure 6-1 ADC Data Point Cloud Processing Flowchart

Disclaimer and Copyright Notice

The information provided by our company in this document is accurate to the best of our knowledge, but we do not guarantee that the content is completely error-free. All liability for any damages arising from the use of this document (including but not limited to indirect, incidental, or special losses) or any improper use resulting from non-compliance with the instructions herein (including but not limited to over-voltage, over-frequency, or over-temperature operations) is explicitly disclaimed.

The content in this document, including URL references, may differ from the actual content due to product version upgrades or other reasons, and is subject to change without notice.

THIS DOCUMENT IS PROVIDED "AS IS" WITH NO WARRANTIES. NO WARRANTY IS PROVIDED TO THIS DOCUMENT FOR ITS MERCHANTABILITY, NON-INFRINGEMENT, FITNESS FOR ANY PARTICULAR PURPOSE, NOR DOES ANY WARRANTY OTHERWISE ARISING OUT OF ANY PROPOSAL, SPECIFICATION OR SAMPLE.

All liability, including liability for infringement of any patent rights, relating to use of information in this document is disclaimed. No licenses express or implied, by estoppel or otherwise, to any intellectual property rights are granted herein.

Copyright © 2026 Possumic Technology Co., Ltd. All rights reserved.